



✉ waterlens@waterlens.moe  github.com/waterlens  waterlens.moe

PERSONAL OVERVIEW

I am dedicated to designing and implementing high-performance programming languages with new paradigms, seeking opportunities for compiler optimizations, and providing users with stronger static safety guarantees.

EDUCATION

Hong Kong University of Science & Technology Sept. 2023 - Jan. 2026 (expected)
MPhil, *Computer Science and Engineering*, Advisor: Lionel Parreaux Hong Kong S.A.R.
Research focus: **compiler optimizations** for functional programming languages.

Zhejiang University Sept. 2019 - June 2023
BEng, *Computer Science and Technology*, GPA: 3.84/4.0 Hangzhou
Courses with A+/A: Compilation Principles, Computer Architecture, Programming Language Principles, Operating Systems, etc.

PUBLICATIONS

Smart Inlining through Function Splitting, *PLDI SRC 2025* Apr. 2025

EXPERIENCE

Intern for Programming Language Tool Development, *IDEA Research, Basic Software Department* Mar. 2025 - June 2025

- Implemented an OCaml optimization to improve MoonBit compiler performance.
- Improved the speed of compiling MoonBit tests with the native backend by using the tcc compiler to compile generated C source, including refactoring the build system, fixing bugs in tcc, and supporting cross-platform runtime libraries for Linux, macOS, and Windows.
- Added benchmarking features to the toolchain, supporting statistical analysis and visualization.

PROJECTS

Calocom  Apr. 2022 - June 2022
Group project for Compilation Principles course Rust

- Developed a feature-rich programming language supporting **algebraic data types**, higher-order functions, and pattern matching.
- Responsible for designing the **type system**, typed AST, object memory representation, name mangling style, and an intermediate representation (IR) for desugaring and necessary transformations.
- Led the project development, implemented almost all components except lexing and parsing, including syntax desugaring, **semantic checking**, **closure conversion**, and **LLVM**-based code generation (using Rust library *inkwell*). Also wrote the standard library (strings, vectors, etc.) and **runtime** (object allocation and program entry) in unsafe Rust.
- Awarded best course project in class.

SyOC  Mar. 2022 - Aug. 2022
Team collaboration project C++, Python, ARM

- Aimed to learn compiler optimization techniques and participate in the Bisheng Cup Compiler Contest. Developed from scratch, first team from Zhejiang University to enter the finals (only two members).
- Led the project, designed a **compiler framework** in **modern C++** for optimization passes, and an **SSA**-based IR supporting **def-use** and **use-def** chains.

- Implemented lexer, recursive descent parser, **mem2reg** transformation (including **immediate dominator analysis** and **iterated dominance frontier analysis** for SSA construction), dead code elimination, and constant propagation.
- Wrote Python scripts to compare the performance of optimized programs with gcc or other compilers.

MLscript

Apr. 2023 - Now

Joint project from lab

Scala, C++

- Designed an ANF-based intermediate representation with join points extension.
- Implemented a **smart inliner** with control flow analysis to decide inlining opportunities, and used **function splitting** to reduce code duplication caused by inlining. Also responsible for the C++ backend, including universal object memory representation, **optimized arithmetic on unboxed values**, and **reference-counting** memory management.

QuickKaml

June 2023 - Jan. 2024

Personal hobby project

C

- Implemented a register-based VM interpreter for a monomorphic language, with many low-level optimizations.
- Used a patched LLVM to generate **guaranteed tail-call** code for VM instruction handlers in the interpreter.
- Tried various techniques to improve interpreter performance, including: **operand reordering** for efficient sign extension; **partial register decoding** to reduce unnecessary shifts on x86-64; instruction fetching based on unaligned memory access or shift and mask; **pre-decoding** to hide decode latency using CPU pipeline concurrency.

MMM

Sept. 2024 - Nov. 2024

Team project for MGPIC contest

MoonBit, RISC-V, WebAssembly

- Led development and designed an optimizing compiler framework based on MoonBit for the *Mini MoonBit* language, with JavaScript, **RISC-V**, and **WASM** backends.
- Implemented all core optimizations required for the contest, including **guaranteed tail recursion elimination**, **selective lambda lifting** based on **register pressure**, **basic block straightening**, dead code elimination, **local value numbering**, **common subexpression elimination**, **loop invariant code motion**, **jump table optimization**, **scalar replacement**, and fast bump allocation.
- Wrote code generators for JavaScript and RISC-V backends. To avoid stack overflow, designed **selective CPS transformation** and **automatic thunking** for the JavaScript backend; for the RISC-V backend, ported Cranelift's **tree-pattern covering instruction selector** and implemented a **chordal graph coloring register allocator**.
- Extended the language to support parametric polymorphism (**generics**), ad-hoc polymorphism (**typeclass** via dictionary passing), and user-defined operators.

RMatch

Sept. 2021 - Oct. 2021

Personal hobby project

C++

- Parsed **regular expressions** and generated NFA-based **VM** bytecode, then used C++ library *xbyak* to **JIT-compile** the bytecode to native x86-64 machine code.

Apple μ Arch Bench

Apr. 2024

Hobby project

C

- Explored **micro-architecture** features of Apple Silicon using **hardware performance counters**.

SIB Optimization for OCaml

May 2025 - June 2025

Share-immutable-block optimization

OCaml

- Functional programming languages often perform pattern matching on existing data structures, and even if the new object is identical to the old one, a new allocation is often made. I implemented a sound optimization that eliminates unnecessary allocations if the object is proven to be immutable.
- This optimization is used internally in the MoonBit compiler.

Monoid Hash

Apr. 2025

Performance-critical part of an incremental computation research project

C, AArch64

- Extended fast-crc32 implementation with ARMv8's `pmull` instruction to accelerate monoid combination. Specifically, this speeds up the multiplication of two bit-reflected polynomials over the $GF(2^{32})$ field.

SKILLS

Programming Languages: Proficient in multiple programming languages, including but not limited to:

- Most frequently used: OCaml, Rust, C/C++, Scala
- Familiar: Java, Python, CUDA C/C++
- Experienced with: C#, TypeScript, JavaScript, Ruby, Haskell, Lua, Verilog, Scheme, etc.

Compilers:

- Experienced in using and modifying common **SSA**-based compiler frameworks such as **LLVM (MLIR)**, Cranelift, etc.
- Proficient in compiling various programming paradigms, including imperative, functional, object-oriented, and dynamic languages.
- Familiar with compiler optimizations such as **sparse conditional constant propagation**, **control flow simplification**, **dead and unreachable code elimination**, **redundancy elimination**, etc.
- Skilled in micro-architecture-level performance tuning using **profiling** tools (e.g., perf, VTune, flamegraph).
- Familiar with multiple **register allocation** algorithms (iterated register coalescing, linear scan, etc.) and **garbage collection** algorithms (mark-sweep, mark-compact, tri-color incremental, generational, etc.).
- Deep understanding of interpreter and runtime system design and implementation, including various threading techniques, stack-based and register-based VMs, memory management, runtime object representation, and **context switching** architectures.

Architecture:

- Designed and implemented a **Tomasulo-based out-of-order RISC-V CPU**.
- Familiar with instruction sets of x86-64, AArch64, RISC-V, etc. Proficient with SSE, AVX, Neon, etc. SIMD instruction sets.
- Proficient in micro-architecture-level performance analysis based on documentation from CPU vendors.

Programming Language Theory:

- Formal verification with Coq.
- Read books on programming languages, including: **Software Foundations**; **Types and Programming Languages**; **Practical Foundations for Programming Languages**; **Essentials of Programming Languages**.
- Familiar with constraint-based type inference, bidirectional type inference, and rich knowledge of type systems.

Languages:

- Chinese (native), English (good working communication)

OTHER EXPERIENCE

Student Volunteer, ICFP 2024

Sept. 2024

Teaching Assistant for C++ Programming

Jan. 2024 - June 2024

- Designed lab sessions to help students understand pointers and references in C++.

Remote Research Intern, Advisor: Yizhou Zhang

Sept. 2022 - Jan. 2023

- Studied the implementation and semantics of **lexical algebraic effects**, a hot topic in programming language research.

Undergraduate Teaching Assistant, Principles of Programming Languages

Sept. 2022 - Jan. 2023

- Designed OCaml-based labs to help students understand the Hindley-Milner **type inference** algorithm.
- Set homework to help students learn and use **evaluation-context-style operational semantics** and **delimited continuations**.
- Designed and implemented an online judge system for the course, utilizing GitHub's CI/CD system and asymmetric encryption.